Machine Learning and Inductive Inference - Lecture 03

Lecture overview

Lecture 03 introduces decision trees as interpretable procedures that map input attributes to target predictions through a sequence of tests, functioning as sets of if-then rules. The lecture distinguishes between classification trees for nominal targets and regression trees for numerical targets, noting that while decision trees can represent any Boolean function, they may require inefficient structures due to an inability to share subtrees. For continuous attributes, the algorithm employs comparative thresholds to create binary splits rather than testing every value. The core learning objective involves approximating an unknown function from training data, balancing the theoretical goal of finding the smallest consistent tree-which is NP-hard-with the practical necessity of minimizing expected risk through heuristics like Top-Down Induction of Decision Trees (TDIDT).

To address overfitting, the lecture contrasts cautious splitting (pre-pruning) with post-pruning, favoring the latter for its ability to construct a full tree and then remove non-contributing branches based on validation set performance. The generic TDIDT framework relies on specific functions for generating tests, selecting the best split via metrics like Information Gain or Gini impurity, and determining stopping criteria. While standard trees create axis-parallel splits that are computationally efficient, oblique trees using linear combinations offer greater expressiveness at a higher computational cost. The session concludes by exploring advanced variations: handling missing values through partial assignment and weighted voting; distinguishing regression trees, which model piecewise constant functions, from model trees (such as M5 and RETIS), which utilize linear models in leaf nodes to approximate piecewise linear relationships more effectively.

Slide 1: Decision trees

This slide introduces the topic of decision trees as the primary subject of the lecture. No additional definitions, examples, or explanatory details are provided in the source material for this specific section. The content serves solely as a title card to establish the focus on decision tree algorithms.

Slide 2: What are decison

A decision tree predicts by testing input attributes along a path from the root to a leaf. Each internal node asks a test, each branch records an outcome, and the leaf supplies the predicted class or value.

Slide 3: What is a decision tree?

A decision tree represents a procedure starting with one question, where each answer determines the next question until a final decision is reached. In this context, questions are termed "tests" and the final decision is a "prediction." Because trees are interpretable, they can be easily represented as a set of if-then rules that summarize the logic used to make these predictions.

Slide 4: Example decision tree

This slide illustrates a decision tree for determining whether to play tennis based on weather conditions. The process begins by testing the outlook: an overcast sky guarantees playing, while sunny conditions require checking humidity (high prevents playing) and rainy conditions require checking wind strength. These sequential tests continue until a final prediction is reached.

Slide 5: What does the tree represent?

The decision tree represents a mathematical function mapping an input space X to a target attribute Y. Given input attributes $X_1, \dots, X_n$, any point in the input space is a vector $\mathbf{x} = (x_1, \dots, x_n)$ that maps to exactly one value for Y. While questions within the tree can involve any input attribute, the final decision assigns a specific predicted value to the target.

Slide 6: Illustration

This decision tree maps inputs Outlook, Humidity, and Wind to a Tennis prediction (Yes/No) across 14 cases. The data splits into Sunny (5 cases), Overcast (4 cases), and Rainy (5 cases), leading to three leaf nodes where two predict "No" and one predicts "Yes." Aggregating these outcomes shows that 10 of the 14 cases result in "No," establishing it as the majority prediction regardless of the specific missing value.

Slide 7: Representation power

Decision trees can represent any imaginable Boolean function by testing all input variables and assigning the correct output for each combination. While simple functions like OR, AND, and XOR are easily represented, complex formulas may require inefficient tree structures because the tree format cannot share subtrees, forcing redundant copies of logic paths. Thus, although representation is always possible, it does not guarantee a compact or efficient model.

Slide 8: Continuous input attributes

When input attributes are continuous, such as numbers between 0 and 1, decision trees cannot create a separate child node for every possible value due to infinite possibilities. Instead, the system employs comparative tests using thresholds to ensure a finite number of outcomes. This approach generates binary splits based on inequalities, directing instances left or right depending on whether the attribute value is less than or greater than a specific threshold like 0.5.

Slide 9: Types of trees

Trees are categorized based on the type of target variable they predict. If the target variable is nominal, the structure is termed a classification tree; if the target variable is numerical, it is called a regression tree. The material notes that additional types of trees exist but will be addressed in later sections.

Slide 10: Why trees?

Decision trees are preferred over alternatives like neural networks primarily for their efficiency and interpretability. They can be learned and used for prediction extremely quickly, often matching the speed of data reading, while remaining transparent enough to explain specific predictions unlike "black box" models. Additionally, they achieve high predictive accuracy on tabular data, which improves further when combined into ensembles.

Slide 11: How can we learn

This slide introduces the fundamental question of how to learn decision trees directly from data. It identifies the existence of a basic learning algorithm designed to address this process, serving as the foundational concept for constructing these models.

Slide 12: Learning decision trees

Predictive learning aims to learn an unknown function $f: X \to Y$ from a dataset of examples $(x, f(x))$. The objective is to find a decision tree $T: X \to Y$ that approximates this function while fulfilling a specific criterion C, the definition of which depends on the task. This process involves extracting a suitable tree structure directly from the training data to represent the underlying functional relationship.

Slide 13: Learning decision trees: task 1

Task 1 in learning decision trees involves finding the smallest tree consistent with the training data, ensuring it predicts every label exactly. This approach acts as data compression and provides insight by revealing the simplest explanation of how inputs relate to outputs. However, this task is primarily of theoretical interest because perfect consistency on a sample does not guarantee accuracy over the larger underlying distribution or unseen cases.

Slide 14: Learning decision trees: task 2

Task 2 in learning decision trees aims to find a tree T where predictions $T(x)$ are maximally similar to true values $f(x)$ on average for inputs drawn from distribution $\mathcal{D}$. This similarity is quantified by a loss function ℓ, measuring prediction error, while the expected loss over the population defines the model's risk. The objective is restated as finding the tree with minimal risk, acknowledging that practical algorithms must approximate this since the true population distribution is unknown.

Slide 15: Learning decision trees: task 2

Since the true distribution $\mathcal{D}$ is typically unknown, computing a decision tree's risk or finding the minimal-risk tree is impossible. Consequently, practical algorithms employ heuristics to construct trees that are small enough to provide insight into the relationship between X and Y, while also generalizing well for predictions outside the training data T.

Slide 16: Hardness of decision tree learning

Complexity theory establishes that learning decision trees is NP-hard, specifically referring to the computationally difficult task of finding the smallest tree consistent with training data. While this optimal search provides guarantees of minimality, practical learners do not attempt it; instead, they use efficient algorithms designed to find trees with low risk on the population rather than the provably smallest one.

Slide 17: Learning DTs: the basic principle

The basic principle of learning decision trees is known as Top-down induction of decision trees (TDIDT) or recursive partitioning. This process starts with the full dataset, finds a test where examples sharing an outcome tend to share the same target value Y, and splits the data into subsets based on that test. The procedure repeats recursively on each subset until they are sufficiently "pure" (all elements have the same Y) or no further splits are possible.

Slide 18: Example: cocktail drinking

This slide illustrates a classification problem where the goal is to predict if a new cocktail will cause sickness based on past experiences. The dataset consists of previous cocktails labeled as either causing sickness or not, with features including glass shape, drink color, and ingredient content. The objective is to determine if any of these properties correlate with the outcome to effectively split positive cases (sick) from negative ones (not sick).

Slide 19: The data set D: 8 examples

The slide identifies dataset D as eight examples. It is the small training set for the following tree-building walkthrough; the displayed cases, rather than a claim about a general population, determine the candidate splits.

Slide 20: Data set in tabular form

Slide 20 presents a data set in tabular form containing eight entries with four attributes: Shape, Color, Content volume, and Sick status. The table lists various combinations of shapes (Cylinder, Coupe, Trapezoid), colors, and volumes (10cl, 15cl, 25cl) to indicate whether the subject is sick. Specifically, three entries marked "Yes" for being sick are all Coupes or Trapezoids with Orange color and either 10cl or 15cl content, while all other combinations are marked "No".

Slide 21: Observation 1: shape is important

Observation 1 establishes that shape is an important factor. The slide explicitly highlights "shape" as the central concept for this observation, though no further definitions, examples, or detailed explanations are provided in the available source material.

Slide 22: Observation 2: Sometimes, color is

Observation 2 states that sometimes color is also important alongside shape. This slide highlights the dual consideration of both shape and color as relevant visual features in specific contexts. No further elaboration or examples are provided in the source material for this observation.

Slide 23: The decision tree

This tree first partitions examples by shape and then uses color in a branch whose examples still need separating. Each root-to-leaf path corresponds to a sequence of tests and a prediction.

Slide 24: Rule representation of tree

Decision trees can be represented as a series of if-then rules that are often simplifiable. The slide illustrates this with two rule sets: one using nested conditions where a cylindrical shape or non-orange color results in a positive outcome, while the other uses a conjunction where a non-cylindrical shape combined with an orange color yields a negative result.

Slide 25: Two main questions…

Following the illustration of the basic principle, two critical implementation details must be addressed. First, a method is required to determine how to select the "best" test for a node. Second, criteria must be established to decide when to stop splitting nodes during the process.

Slide 26: How to choose the tests?

Choosing optimal tests is analogous to the "20 questions" game, where effective binary questions should eliminate approximately 50% of cases regardless of the answer. Asking a balanced question like gender is superior to specific queries, such as naming Dolly Parton, which likely yield no information if answered negatively. This constraint ensures fairness by preventing open-ended questions that could disproportionately reduce the search space.

Slide 27: Choosing a test: classification

This slide focuses on classification trees where the target variable is nominal. A good test is defined as one that carries significant information about the class. Information theory quantifies this using "entropy," or missing information, which represents the average number of bits required to convey a piece of information under optimal encoding.

Slide 28: Information

To encode four classes (A, B, C, D), fixed-length codes require 2 bits per object, which is optimal only if all classes are equally likely. When frequencies vary, variable-length encoding assigns shorter bit sequences to frequent classes and longer ones to rare classes, provided the code remains unambiguous. For instance, with A at 50% frequency, an optimized scheme reduces the average requirement to 1.7 bits, whereas equal probabilities result in a less efficient average of 2.25 bits using variable lengths.

Slide 29: Information

Entropy represents the inherent information content and the minimal average number of bits required to encode a randomly drawn value from a set with known probabilities. It is calculated by summing the product of each class proportion and its binary logarithm across all classes, then applying a negative sign to ensure a positive result. This metric defines the optimal encoding efficiency based on the distribution of values.

Slide 30: Class entropy

Class entropy quantifies the uncertainty regarding the class of a particular instance within a set of objects S. It is calculated using the formula $CE(S) = -\sum_{i=1}^{k} p_i \log_2(p_i)$, where p_i represents the proportion of elements in S belonging to class c_i. This metric aggregates the distribution of k possible classes to determine the overall impurity or unpredictability of the dataset.

Slide 31: Entropy: example

Entropy measures uncertainty in class prediction, calculated as the negative sum of class proportions multiplied by their log base 2 probabilities. High entropy occurs when many possibilities are equally likely (e.g., 8A and 8B yields CE=1), while low entropy arises when few outcomes exist or one is highly probable (e.g., 15A and 1B yields CE=0.34). This metric quantifies the bits of information needed to identify a random element's class, ranging from maximal uncertainty in balanced distributions to minimal uncertainty in skewed ones.

Slide 32: Information gain

Entropy reflects missing information, quantifying the uncertainty within a dataset. Information gain measures the expected reduction in this entropy when answering a question, serving as a metric for how informative a split is in classification trees. It is calculated as the difference between the original class entropy and the weighted average of the class entropies resulting from the split subsets.

Slide 33: Example

This tennis example analyzes a dataset of 14 cases (9 positive, 5 negative) by calculating information gain for Humidity and Wind attributes. Splitting by Humidity yields subsets with entropies of 0.985 and 0.592, resulting in an information gain of 0.151. In contrast, splitting by Wind produces a lower gain of 0.048, demonstrating that Humidity provides more useful information for classification than Wind in this specific scenario.

Slide 34: Choosing a test: regression

When the target variable is numerical, regression tests are chosen by evaluating how well a split minimizes prediction error. Since predictions typically use the mean of the subset values, the optimal test creates groups where instances have similar values, ensuring the mean closely approximates the actual data points. This reduces average loss by preventing predictions from deviating significantly from real values.

Slide 35: Variance reduction

Variance reduction serves as a heuristic for decision trees when dealing with continuous values, aiming to create subsets where examples have similar values by minimizing spread rather than maximizing information gain. It is calculated as the original set's variance minus the weighted average variance of the resulting subsets, where greater reduction indicates better splits. Maximal reduction occurs when all values in the resulting subsets are identical, yielding a variance of zero.

Slide 36: When to stop splitting nodes?

In principle, node splitting continues until every subset contains instances with identical Y values, creating pure leaves. While this approach is optimal for Task 1 where generalization is unnecessary, it poses a significant risk of overfitting in Task 2. Consequently, stopping the splitting process earlier than achieving complete purity is often required to ensure better model performance on new data.

Slide 37: 37

Slide 37 illustrates a decision tree structure where internal nodes perform binary tests in the form of $X < \dots$ or $Y < \dots$. The visual representation uses sequences of plus and minus signs to denote the resulting classification outcomes along different branches. This diagram demonstrates how splitting on variables X and Y partitions the data space into distinct regions based on these threshold comparisons.

Slide 38: 38

Slide 38 illustrates a decision tree structure that utilizes comparison tests in the form of $X < \dots$ or $Y < \dots$. The specific example provided demonstrates a node evaluating the condition $Y < 4$, which branches into "yes" and "no" outcomes based on the variable values. This visual representation highlights how such trees partition data using simple inequality thresholds.

Slide 39: 39

Slide 39 illustrates a decision tree that classifies data points as positive (+) or negative (-) using tests of the form $X < \dots$ or $Y < \dots$. The specific example shows a root node testing $Y < 4$, which branches to a "yes" outcome (positive) and a "no" path leading to a secondary test of $X < 3$. This structure demonstrates how sequential threshold comparisons on variables X and Y partition the feature space into distinct classification regions.

Slide 40: 40

This slide illustrates constructing a decision tree using two input variables, X and Y, where tests take the form of inequalities like $Y < 4$ or $X < 3$. The first split at $Y < 4$ effectively separates most negative instances from the rest, while a subsequent split at $X < 3$ further refines the classification. Although this tree structure successfully groups the data points, the professor notes that it remains imperfect because it does not achieve a completely pure separation of positive and negative outputs.

Slide 41: Overfitting: effect on generalization

Overfitting increases a decision tree's consistency with the training dataset (D) but reduces its accuracy on instances outside that set. As the tree size grows, performance on the training data improves while generalization to unseen data declines, marking the onset of overfitting where the model fails to generalize effectively beyond the specific examples it was trained on.

Slide 42: How to avoid overfitting?

To avoid overfitting in decision trees, two primary strategies are employed. The first is "cautious splitting," which restricts node division unless the split is confirmed to be meaningful and unlikely to cause overfitting. The second approach is "post-pruning," where a large tree is constructed without immediate concern for overfitting, followed by the removal of branches that provide little contribution.

Slide 43: 1. Cautious splitting

Cautious splitting addresses when to stop growing a decision tree to avoid overfitting, which occurs when accuracy on training data increases while accuracy on unseen data declines. The goal is to identify the "turning point" where generalization performance peaks before it drops due to excessive tree size. Although using predicted accuracy directly as a splitting criterion is ineffective, practical approaches involve setting minimum instance thresholds for subtrees rather than splitting until perfect separation is achieved.

Slide 44: 44

To estimate the turning point when performance curves rely on unseen data, a validation set is employed. This approach involves setting aside a specific subset of data that is excluded from the tree-growing process. Instead, this reserved set is used exclusively to evaluate accuracy on unseen data, providing an unbiased estimate of model performance.

Slide 45: 2. Post-pruning

Post-pruning addresses the risk of stopping tree growth too early due to temporary accuracy dips by first growing the tree to its full size. The algorithm then evaluates nodes starting from the bottom, calculating validation set accuracy if each branch were removed. If removing a node does not lower accuracy, that branch is cut, resulting in a smaller tree with comparable or improved predictive performance.

Slide 46: Illustration

Illustration 46 demonstrates pruning subtrees in a specific sequence, removing nodes A, D, C, and others only when accuracy remains unchanged. The slide presents this ordered process alongside a numerical list from 1 to 7 and explicitly questions why the order of pruning is important for maintaining model performance.

Slide 47: 47

The plots compare tree size, training accuracy, unseen-data accuracy, and the effect of pruning. A larger tree can fit training cases more closely but may generalize worse; pruning can reduce complexity and sometimes improve unseen accuracy. Read the plotted trade-off rather than assuming more nodes always improve prediction.

Slide 48: Post-pruning vs. cautious splitting

Modern tree learners prefer post-pruning over cautious splitting (pre-pruning) because, despite requiring more effort to build and then reduce a large tree, it yields slightly more accurate results. Since tree learning is highly efficient, the additional computational cost is generally acceptable. Conversely, failing to prune a tree at all is typically inadvisable as it leads to overfitting.

Slide 49: A generic algorithm

Slide 49 introduces the concept of a generic algorithm, highlighting the common structural framework shared by Decision Tree (DT) learners. It further notes that while these learners share this foundational structure, distinct differences exist between specific implementations. No additional details regarding the algorithm's mechanics or specific variations are provided in the source material for this section.

Slide 50: A generic TDIDT algorithm

TDIDT stands for "top-down induction of decision trees," also known as recursive partitioning. While most learners follow this basic approach to efficiently create small, low-risk trees, they differ in specific details. The process typically involves two phases: first, growing the tree by adding and splitting nodes, followed by a second phase of pruning.

Slide 51: A generic TDIDT algorithm

The generic TDIDT algorithm constructs a decision tree by first growing a full tree and then pruning it. The recursive `grow_tree` function selects the best test to partition examples into subsets; if a stopping criterion is met, it returns a leaf node, otherwise it recursively builds subtrees for each outcome. Functions like `generate_tests` allow for variations in how tests are created based on attribute types, such as nominal attributes with multiple outcomes.

Slide 52: A generic TDIDT algorithm

A generic Top-Down Induction of Decision Trees (TDIDT) algorithm relies on five key functions where specific implementations differ. These include `prune_tree` for post-construction pruning, `generate_tests` to identify candidate tests, and `best_test` to select the optimal test using heuristics. Additionally, the algorithm defines a `stop_criterion` to determine when splitting ceases and specifies `info` for data stored in leaf nodes.

Slide 53: generate_tests

While standard decision trees create one subtree per nominal value or two subtrees for numerical attributes using $X<c$, alternatives exist for complex scenarios. For nominal attributes with many values, splits can group values into sets to produce binary outcomes. Numerical attributes offer further flexibility through interval tests ($X \in [c_1, c_2]$) or linear combinations ($aX_1 + bX_2 < c$), alongside the critical challenge of determining the optimal threshold constant c.

Slide 54: Illustration: effect of type of tests

Slide 54 illustrates how different test types affect decision boundaries using a dataset of positive and negative instances. It contrasts "oblique trees," which utilize linear combinations of features in the form $aX+bY<c$, against standard axis-parallel tests defined by individual thresholds like $X<c$ or $Y<c$. This comparison highlights the geometric differences between diagonal splits and orthogonal splits when separating classes.

Slide 55: Type of tests affects expressiveness

Oblique trees are more expressive than non-oblique trees because they use linear combinations of variables to create diagonal class boundaries, often requiring fewer splits to separate data. However, they are significantly harder to learn as determining the optimal test involves optimizing multiple parameters rather than a single threshold. Consequently, despite their theoretical advantages in specific cases, non-oblique trees remain much more common in practice due to their faster construction and lower computational cost.

Slide 56: best_test

For classification trees, best_test selection typically uses information gain (entropy reduction) or Gini impurity reduction, while regression trees rely on variance or standard deviation reduction. The professor illustrates that a split yielding identical class distributions in child nodes results in zero information gain, whereas a split creating at least one pure leaf reduces entropy and provides positive gain. Algorithms like C4.5 use gain ratio as an alternative to information gain, while CART specifically minimizes Gini impurity.

Slide 57: • Tests with many possible outcomes have an (unfair?) advantage over tests with few

Tests with many possible outcomes possess an unfair advantage in decision trees because they can generate higher information gain than binary tests. To address this bias, which can lead to overfitting, the Gain Ratio normalizes Information Gain by dividing it by "Split Information," a measure of the test's maximum potential gain based on outcome distribution. This heuristic ensures fairer comparisons between attributes with varying numbers of values.

Slide 58: • Typically a binary split of the form x < c is made

For numerical attributes, a binary split of the form $x < c$ is typically created by testing multiple candidate values for the threshold c. The optimal c is selected as the one yielding the best heuristic value, such as Information Gain or Gini index. Although evaluating many potential thresholds appears computationally expensive, efficient methods exist to determine this optimal split with performance comparable to handling nominal data.

Slide 59: info

For classification trees, leaves typically store the most frequent class, though they may alternatively retain the full class distribution or all relevant training examples to support uncertainty assessment. In regression trees, the leaf value is usually the mean of target values, with alternatives including the median or variance. These storage options allow users to decide prediction thresholds based on confidence levels rather than relying solely on a single point estimate.

Slide 60: stop_criterion

Although post-pruning is generally preferred over early stopping, specific criteria justify halting tree growth immediately. These include cases where all instances share the same class or when too few examples remain to split further. To ensure meaningful generalization and avoid leaves of size one, splitting should only proceed if tests yield subtrees containing at least two examples each.

Slide 61: A note on heuristics for classification

While C4.5 uses entropy and CART uses Gini as impurity measures, using predictive accuracy directly as a heuristic fails because it does not distinguish between splits that improve class purity versus those that do not. As demonstrated by comparing two split scenarios, accuracy can remain unchanged even when one split creates a perfectly pure node, rendering it ineffective for guiding tree construction. Consequently, established impurity metrics like Gini and Information Gain are preferred over simple accuracy to ensure meaningful test selection.

Slide 62: Impurity measures

Impurity measures quantify uncertainty as a function of class proportion p in binary classification, where entropy reaches zero at pure nodes and peaks at a 50-50 split. Classification error, defined as one minus accuracy, decreases linearly from the majority class proportion until reaching 0.5 before rising again. A key observation is that effective impurity measures must be strictly concave, exhibiting curved behavior rather than straight lines to properly reflect uncertainty changes.

Slide 63: Good impurity measures are strictly

Good impurity measures must be strictly concave functions, as established by Breiman et al. (1984). Information gain is calculated as the vertical difference between the original entropy point at proportion p and the weighted average entropy of the split nodes, which lies on the straight line connecting the child nodes' entropies. Because accuracy or error measures form a straight line rather than a concave curve, they yield zero information gain, making them ineffective for determining optimal splits.

Slide 64: Good impurity measures are strictly

Good impurity measures must be strictly concave. In contrast, accuracy is often too restrictive because it only improves if a child node's majority class differs from its parent's; otherwise, many valid tests yield zero accuracy gain. This limitation arises because linear interpolation between points results in no gain, failing to capture the benefits of splitting when the majority class remains unchanged.

Slide 65: Good impurity measures are strictly

Good impurity measures must be strictly concave to ensure consistent improvement. In contrast, accuracy only yields a non-zero gain if a split changes the majority class of at least one child node relative to its parent. This requirement is often too restrictive because small, incremental improvements in purity result in zero accuracy gain, making metrics like Information Gain or Gini preferable for decision tree construction.

Slide 66: How much work is it?

Learning and using a decision tree have different costs. Learning must search candidate tests and tree structures from data; prediction follows one path through an already-built tree, usually requiring a number of tests proportional to tree depth.

Slide 67: Computational complexity of tree

This section analyzes the computational complexity of tree learning by examining how dataset size influences effort. Given a dataset D with N instances and m attributes, the analysis investigates how increasing rows (N) or columns (m) affects the work required to learn a tree. The discussion specifically focuses on determining the best test for a node, noting that computing thresholds for numerical variables represents significant computational work despite the overall efficiency of the learning process.

Slide 68: Splitting one node

To split a node, the algorithm identifies the best test by evaluating every possible candidate. For each test, it partitions the dataset and computes a quality metric, such as entropy, to determine its score. The test yielding the highest quality is selected, and the data is finally repartitioned according to this optimal split before proceeding to the next node.

Slide 69: Computing the quality of one test

To compute the quality of a single test, use information gain, defined as the reduction in entropy. This requires calculating class entropy for the parent node and its subsets by determining the proportion (p_i) of each class within those sets. The proportion is found by counting instances of a specific class in a subset and dividing by the total number of instances in that set.

Slide 70: Computing the quality of one test

Slide 70 demonstrates computing the quality of a single test using a dataset with attributes A, B, and C against positive and negative classes. The slide presents counts for each attribute's distribution across classes, noting that for attribute C, the probability of being positive (ppos) is 0.5 and the probability of being negative (pneg) is 0.5. This calculation relies on the provided frequency table where attribute C appears once in the positive class and once in the negative class out of two total instances.

Slide 71: Computing the quality of one test:

To efficiently evaluate continuous attribute thresholds, start with a threshold below all values where no cases satisfy the condition. As the threshold increases to the next value, update the contingency table by moving only the single affected case from the "no" column to the "yes" column. This incremental update avoids recalculating the entire table for every possibility, significantly reducing the computational work required to assess test quality across all thresholds.

Slide 72: Efficiently computing test quality for

To efficiently compute test quality for continuous attributes, first sort all tuples by the attribute value from smallest to largest. By gradually moving the threshold and simply updating the class count tables, testing all possible thresholds requires only slightly more work than testing a single one. This incremental approach avoids redundant calculations while evaluating split points.

Slide 73: Efficiently computing test quality for

To efficiently compute test quality for continuous attributes, first sort all tuples by the attribute value from small to large. By gradually moving the threshold and simply updating the count tables, testing all possible thresholds requires only slightly more work than testing a single one. This approach avoids redundant calculations while evaluating split points like A < 2.5.

Slide 74: Efficiently computing test quality for

To efficiently compute test quality for continuous attributes, first sort all tuples by the attribute value from smallest to largest. By gradually moving the threshold through this sorted list, class count tables can be updated incrementally rather than recalculated from scratch. This approach ensures that testing all possible thresholds requires only slightly more computational effort than evaluating a single threshold.

Slide 75: Computing the quality of one test

Computing test quality requires a single scan of the dataset for both nominal and continuous attributes. For nominal attributes, Information Gain (IG) is calculated once after building the table, whereas continuous attributes require sorting the data initially and then updating the table to compute IG for each intermediate split during one pass. Although continuous attributes involve more calculations, this adds only a constant factor, ensuring that finding the best test for a node remains linear in the number of instances.

Slide 76: Splitting one node

When splitting a single node containing n instances (distinct from the total dataset size N), the computational work scales linearly with n. Since the number of tests is typically proportional to the number of attributes m, and computing the quality of each test requires $O(n)$ time, the total complexity to evaluate all tests is $O(nm)$. This overall workload corresponds directly to the size of the data table for that specific node.

Slide 77: Splitting multiple nodes

Splitting a node increases the number of nodes but reduces the data elements in each, keeping the total instances per tree level constant at N. Consequently, the overall work remains the same for each level because processing time scales linearly with instance count; for example, splitting 10,000 instances into 2,000 and 8,000 results in combined processing times of 0.2ms and 0.8ms, totaling the original 1ms. This work only diminishes when leaf nodes are created.

Slide 78: Overall complexity

If decision tree splits are balanced, the tree height is O(log N), resulting in an overall learning complexity of O(m N log N), where N is the number of instances and m is the number of attributes. This makes tree growing nearly linear relative to dataset size because the logarithmic factor grows slowly. However, if splits are unbalanced, the complexity can degrade to $O(m N^2)$.

Slide 79: Overall complexity: unbalanced trees

In unbalanced decision trees, the height can reach O(N), a scenario that is possible in practice when splits are highly skewed. However, effective heuristics like Information Gain (IG) generally promote balanced splits to avoid this worst-case complexity. The slide illustrates how uneven instance distribution at each node contributes to this linear growth in tree depth.

Slide 80: Making predictions

Making predictions with decision trees is highly efficient, requiring only h tests where h is the tree height. Heuristics like information gain create balanced splits that minimize height by rapidly reducing entropy or variance to reach pure leaf nodes. Consequently, a single prediction typically involves just 5-6 tests, enabling millions of cases to be processed per second.

Slide 81: Complexity: conclusions

Under mild assumptions, decision tree learning is highly efficient, making it particularly advantageous for processing big data. Additionally, prediction using decision trees is extremely fast, typically completing in less than one microsecond. These properties highlight the algorithm's scalability and speed in practical applications.

Slide 82: Handling missing

Slide 82 introduces the topic of handling missing values in data. The provided source material consists solely of the slide title and contains no further definitions, methods, examples, or professor explanations to elaborate on this concept. Consequently, no specific techniques or details regarding the management of missing data can be derived from the available text for this section.

Slide 83: What about missing values?

When computing test quality, ignore instances with missing values for that specific attribute while retaining them for other tests. During splitting, handle missing values by either imputing the most likely value or partially assigning the example to multiple branches proportional to their probabilities. For prediction, generate a weighted vote among all leaves where the example (partially) ends up based on these branching possibilities.

Slide 84: Distribution: illustration

Slide 84 illustrates a classification problem using a dataset of weather attributes (Outlook, Humidity, Wind) to predict whether to "Play." It demonstrates calculating class probabilities for a new instance [?, high, strong], where the distribution shows 10/14 instances as "no" and 4/14 as "yes." Based on this majority vote, the slide concludes that the predicted classification for the example is "no."

Slide 85: Model trees

Model trees are defined as more complex versions of regression trees. This slide introduces them as an advanced structure that builds upon the foundational concepts of regression trees, though no specific mechanisms or examples are provided in the source material.

Slide 86: Representation power of

Regression trees possess the representation power to model piecewise constant functions. As illustrated by the decision structure involving thresholds at $X < 1$ and $X < 2$, these trees partition the input space into distinct regions where the output Y remains constant, such as values of 0.5, 1.5, or 2 depending on the specific path taken through the tree nodes.

Slide 87: Representation power of

Slide 87 illustrates the representation power of regression trees by attempting to model the linear target function Y=X. The slide depicts a tree structure using split conditions like X<0.5 and X<1 to produce constant predictions (e.g., Y=0.25, Y=0.75) within specific intervals. This demonstrates that regression trees approximate continuous functions through piecewise constant segments rather than capturing the linear relationship directly.

Slide 88: Model trees

Model trees are a variation of regression trees where each leaf node contains a linear model rather than a single constant value. This structure allows the tree to represent piecewise linear functions, enabling it to fit complex relationships that standard piecewise constant regression trees cannot capture effectively.

Slide 89: A model tree learner: M5

M5, proposed by Quinlan (1992) and implemented as M5' in Weka (Wang & Witten, 1997), is a prominent model tree learner. It performs standard regression tree learning but replaces leaf values with linear models once tree construction concludes. Although M5 selects tests to minimize variance, the slide notes this criterion is no longer optimal for this specific approach, prompting consideration of better alternatives.

Slide 90: RETIS

A subset is considered good if its data can be well approximated by a linear model, noting that Y variance is not necessarily a reliable indicator of this quality. Consequently, M5's heuristics are appropriate for regression trees with constant leaf predictions but fail when leaves utilize linear models. The splitting procedure aims to find tests where subsets share similar target values, repeating until all subsets contain identical Y values.

Slide 91: RETIS

The RETIS system (Karalic, 1992) improves subset splitting by performing linear regression in each potential subset and selecting the test that minimizes the average residual variance. However, determining the optimal test has a computational complexity of $O(m^3)$, where m is the number of variables, due to the requirements of multiple regression. Consequently, RETIS becomes computationally prohibitive and very slow when applied to datasets with a high number of variables.

Slide 92: Mauve

Mauve is a model tree learner that selects tests by evaluating the reduction of residual variance after univariate linear regression ($Y=aX+b$). It produces smaller and more accurate trees than M5, with performance speed slightly slower than M5 but faster than RETIS. This approach is based on the heuristic proposed by Vens et al. (2006) for learning model trees.

Slide 93: Mauve: motivating example

Slide 93 presents a motivating example for Mauve using a target function defined as f(x)=x for 0<x<1 and f(x)=1-x for 1<x<2. The slide notes that a decision tree exists to express this function perfectly, specifically splitting on the condition X<1 to yield Y=X or Y=1-X. It then poses the critical question of whether the M5 algorithm can successfully identify this optimal tree structure.

Slide 94: Mauve: motivating example

Slide 94 presents a motivating example demonstrating that minimizing average variance does not necessarily yield an optimal split in decision trees. The provided data shows that while the average variance is maximally reduced at a threshold of approximately $c \approx 0.4$, it fails to decrease for $c=1$, which represents the actual optimal test. This discrepancy highlights the limitation of using average variance minimization as a sole criterion for determining split quality.

Slide 95: Mauve: motivating example

Slide 95 presents a motivating example where the M5 algorithm constructs a complex decision tree to model scattered data points. The resulting structure features ten leaf nodes (LM1-LM10), each defined by specific linear regression equations or constant values derived from split conditions such as $x \leq 1.6$ and $x \leq 0.51$. This visualization demonstrates how M5 partitions the input space to fit local linear models across different regions.

Slide 96: Mauve vs. M5’

Slide 96 presents an experimental comparison of Mauve and M5' on synthetic data, visualized through multiple plots. The slide includes axis labels ranging from 0.0 to 2.0 and 0.0 to 1.2, alongside a legend identifying "Mauve," "M5'," and "Random splits." No specific performance metrics, trends, or conclusions are provided in the source text or transcript for this section.

Slide 97: Multi-target trees

Multi-target trees are a method for predicting multiple variables simultaneously using a single decision tree. This approach consolidates the prediction of several target outputs into one unified model structure, rather than building separate trees for each variable. The core concept focuses on efficiency and integration within the tree-based learning framework.

Slide 98: Multi-target trees

While classification and regression trees typically predict a single target variable, the principle of variance reduction can be generalized to handle vector predictions. This extension enables a single tree to simultaneously predict multiple variables. The slide concludes by posing the critical question of whether this multi-target approach performs effectively in practice.

Slide 99: Multi-label classification

Multi-label classification involves predicting a set of labels for each instance rather than a single class. A primary example is assigning multiple keywords to texts, where the input X represents the text description (e.g., bag-of-words) and the output Y is a set of relevant keywords like {animals, climate change, North Pole}. The core challenge addressed is determining how to learn a model capable of predicting these label sets.

Slide 100: Multi-label classification

Multi-label classification employs two primary approaches: binary relevance and label powersets. Binary relevance trains one independent binary decision tree for each individual label to predict yes/no outcomes. In contrast, label powersets treats every possible combination of labels as a distinct class, resulting in 2^n combined labels from n original labels, which are then predicted by a single decision tree.

Slide 101: Multi-label classification

Multi-label classification can be implemented in decision trees using vector encoding, where each label set is represented as a binary 0/1 vector. This approach requires minimal algorithmic changes: the split criterion uses the variance of these vectors instead of scalar targets, and leaf nodes store the mean vector representing class proportions. For instance, averaging multiple binary vectors in a leaf yields a probability vector like [1, 1, 0, 0.67, 0.33].

Slide 102: Multi-label trees: prediction

Multi-label trees predict sets of labels rather than a single label by locating the leaf for a new instance and selecting all labels where the proportion exceeds a given threshold, typically 0.5. For example, a leaf distribution of [1, 1, 0, 0.67, 0.33] yields the predicted set {a, b, d}. This approach allows one tree to simultaneously make multiple predictions, such as for protein functions, often outperforming separate trees for each label.

Slide 103: Hierarchical multilabel classification

Hierarchical multilabel classification extends standard multilabel problems by organizing labels into a hierarchy where a label can only appear if all its ancestors are also present. This constraint ensures that predictions respect the structural relationships between categories. Common applications include classifying texts into newsgroups and assigning functions to proteins.

Slide 104: Hierarchical multilabel classification

Hierarchical multilabel classification with decision trees utilizes the same vector encoding approach as regular multilabel classification. This method automatically satisfies the constraint that a label can only appear if all its ancestors are present in the instance's label set. Additionally, weights can be assigned to different labels to penalize high-level mistakes more strongly than low-level ones.

Slide 105: An application & empirical validation

This slide presents an application of Hierarchical Multilabel Classification (HMC) in functional genomics to predict protein biological functions from their descriptions. The task involves distinguishing among 250 hierarchically organized functions, where a single protein can possess multiple roles in intracellular processes. This case study is based on Blockeel et al. (2006), which utilizes decision trees for this specific classification challenge.

Slide 106: “Normal” classification trees

While "normal" classification trees are advantageous for being fast and interpretable, learning a separate tree for each function undermines these benefits. In scenarios with 250 functions, generating 250 distinct trees results in a system that is no longer fast or interpretable. This approach contrasts the efficiency of single-tree models with the complexity introduced by scaling to multiple descriptive attributes and labels.

Slide 107: HMC trees

HMC trees learn a single decision tree to predict a 250-dimensional class vector representing multiple labels. For heuristic calculations, "variance" is defined as the mean squared distance to the mean within this 250-dimensional space. The model maps descriptive attributes (A1 through An) to specific label combinations, such as instances G1 and G2 predicting distinct subsets of the 250 possible classes.

Slide 108: Good interpretability

Converting decision trees into if-then rule format yields short, interpretable rules. For instance, a specific set of conditions involving nitrogen depletion and stress factors predicts classes such as 40/3 with high precision (0.97), meaning 97% of cases meeting these criteria are correctly classified. However, the recall is low at 0.15, indicating this rule covers only 15% of all actual class 40/3 instances.

Slide 109: Decision trees in

This page opens the decision-trees-in-practice section. It shifts from the tree-learning ideas to their use in applied work; the title alone does not establish a particular application result.

Slide 110: Weka

Weka is a popular, free, and easy-to-use data mining tool available for download at http://www.cs.waikato.ac.nz/ml/weka/. It includes highly optimized learners such as J48 for decision trees and M5' for model trees. These algorithms are variants of the C4.5 and M5 methods, respectively.

Slide 111: Python: scikit-learn

Python's scikit-learn package is widely used in machine learning and provides basic decision tree support. The slide demonstrates loading the Iris dataset, training a `DecisionTreeClassifier`, and visualizing the resulting tree using `matplotlib` and `tree.plot_tree`. This workflow illustrates how to load data, fit a model, and plot the decision structure directly within Python.

Slide 112: Python: scikit-learn

Slide 112 introduces the topic of using the scikit-learn library within the Python programming environment. As no accompanying transcript or detailed slide text is provided for this specific section, no further definitions, examples, or technical explanations regarding its functionality can be derived from the available sources.

Slide 113: Python: scikit-learn

This page introduces scikit-learn's DecisionTreeClassifier API and the methods listed on the slide. In a typical workflow, fit trains the classifier on labeled examples and predict applies it to new inputs; use only the specific methods shown here rather than assuming the slide documents every API detail.

Slide 114: Other software

Many decision tree learners are available as stand-alone applications or integrated within data analysis platforms like R, SAS, and SPSS. Consequently, almost all modern statistical software packages now include decision trees as a standard analysis method.

Key points to remember

To address overfitting, the lecture contrasts cautious splitting (pre-pruning) with post-pruning, favoring the latter for its ability to construct a full tree and then remove non-contributing branches based on validation set performance. The generic TDIDT framework relies on specific functions for generating tests, selecting the best split via metrics like Information Gain or Gini impurity, and determining stopping criteria. While standard trees create axis-parallel splits that are computationally efficient, oblique trees using linear combinations offer greater expressiveness at a higher computational cost. The session concludes by exploring advanced variations: handling missing values through partial assignment and weighted voting; distinguishing regression trees, which model piecewise constant functions, from model trees (such as M5 and RETIS), which utilize linear models in leaf nodes to approximate piecewise linear relationships more effectively.